

Sahara v2 — Backend Architecture

Complete Technical & API Handover Guide for Frontend Development

Backend: FastAPI + SQLite

AI Engine: Gemini 2.0 / 3.6 Flash

Default URL: http://127.0.0.1:8000

1. Core Privacy Architecture & Contract

Strict Privacy Principles Implemented in v2:

- **No Shared Anon Identity:** Every client must maintain a unique UUID (`sess-<uuid>`). The `"anon-session"` fallback is completely removed to eliminate session collisions.
- **Silent Tag Inference:** Triage companion AI infers distress tags strictly from a 7-term controlled vocabulary. Stored internally on session, never shown raw to peer Saathis.
- **Saathi Alias Shielding:** Students only interact with auto-generated peer aliases (e.g. `NightOwl_Eng_23`). Real names are restricted to authenticated admins.
- **Dual-Saathi Model:** Server matches 2 peer supporters simultaneously (`PRIMARY` and `SECONDARY`). Student can switch focus with zero justification.
- **Decoupled Clinical PII:** Professional counselling appointments (`student_name` , `student_email`) are stored in an isolated table with no foreign keys to chat logs.

2. Distress Triage & Tag Vocabulary

Severity Classifications

Severity	Trigger / Meaning
SEVERE	Self-harm, crisis keywords. Surface 24/7 helplines immediately.
MODERATE	Emotional fatigue, isolation. Offer Saathi peer connection.
MILD	Normal exam/study stress. Suggest breathing exercises.

Controlled Inferred Tags

Extracted purely from dialog context, zero free text:

ACADEMIC_STRESS	EXAM_PERIOD	SLEEP_ISSUES
NIGHT_OWL	FAMILY_STRESS	
RELATIONSHIP_STRESS	ISOLATION	

3. Complete API Endpoint Reference

POST /api/chat

AI Companion Triage & Chat

Sends student message to empathetic Sahara companion. Returns AI reply, severity level, and inferred tags.

Request Body:

```
{
  "session_id": "sess-9b1deb4d",
  "userMessage": "Exams are next week and I cannot sleep.",
  "currentSeverity": "MODERATE"
}
```

Response (200 OK):

```
{
  "session_id": "sess-9b1deb4d",
  "reply": "Exam pressure can create a vicious cycle. Let's talk about it.",
  "suggestedSeverity": "MILD",
  "inferredTags": ["academic_stress", "sleep_issues"],
  "source": "langchain-gemini"
}
```

POST /api/saathi/match

Intelligent Dual-Saathi Matching

Matches 2 Saathis based on session tags with capacity cap enforcement and random jitter. Returns aliases and intros.

Request Body:

```
{
  "session_id": "sess-9b1deb4d"
}
```

Response (200 OK):

```
{
  "session_id": "sess-9b1deb4d",
  "primary": {
    "chat_id": "schat-89496809",
    "saathi_id": "saathi-arjun",
    "alias": "Pacer_Comm_22",
    "role": "PRIMARY",
    "status": "ACTIVE",
    "intro_message": "Hey! I'm Pacer_Comm_22...",
    "vibeTags": [{"icon": "🏀", "label": "Sports"}]
  },
  "secondary": {
    "chat_id": "schat-782cea20",
    "saathi_id": "saathi-rohan",
    "alias": "SeniorCode_IT_21",
    "role": "SECONDARY",
    "status": "ACTIVE",
    "intro_message": "Hi there, I'm SeniorCode_IT_21..."
  }
}
```

Swaps UI active focus between Primary and Secondary peer supporters without justification or notifications.

Request Body:

```
{
  "session_id": "sess-9b1deb4d",
  "target_role": "SECONDARY"
}
```

Response (200 OK):

```
{
  "session_id": "sess-9b1deb4d",
  "active_chat_id": "schat-782cea20",
  "active_role": "PRIMARY",
  "active_saathi_alias": "SeniorCode_IT_21",
  "message": "Active peer focus switched successfully with zero guilt."
}
```

Enforces 2-week notice period. Copies chat history to replacement Saathi only if

`consented_history_transfer = true`.

Request Body:

```
{
  "saathi_chat_id": "schat-89496809",
  "notice_weeks": 2,
  "consented_history_transfer": true
}
```

Response (200 OK):

```
{
  "saathi_chat_id": "schat-89496809",
  "status": "TRANSITIONING",
  "transition_notice_date": "2026-09-22",
  "consented_history_transfer": true,
  "new_chat_id": "schat-ce2443f4",
  "new_saathi_alias": "QuietAnchor_Des_22"
}
```

Lets Saathi flag targeting/recognition risk. Silently assigns a replacement without notifying student.

Request: { "saathi_chat_id": "schat-782cea20" }

Response: { "reassigned": true, "new_chat_id": "schat-953e0361", "new_saathi_alias": "NightOwl_Eng_23" }

Consented student name and email stored strictly isolated from chat history and session UUIDs.

Request Body:

```
{
  "counsellor_id": "counsellor-1",
  "counsellor_name": "Dr. Ananya Sen",
  "student_name": "Kavya S.",
  "student_email": "kavya@campus.edu",
  "modality": "Video",
  "selected_slot": "Tomorrow, 3:00 PM"
}
```

Response (200 OK):

```
{
  "id": "apt-6bbb2c75",
  "counsellor_id": "counsellor-1",
  "student_name": "Kavya S.",
  "modality": "Video",
  "status": "CONFIRMED",
  "created_at": "Sep 08, 2026 11:22 AM"
}
```

4. Complete Database Schema (SQLite)

Table Name	Key Columns	Privacy / Security Role
sessions	session_id (PK), created_at, last_active, inferred_tags	Anonymous state & inferred tags only. Zero PII.
chat_messages	id (PK), session_id, sender, text, severity	Sahara AI companion message threads.
checkins	id (PK), session_id, mood, energy, stressor, note	Daily self-reported mood check-ins.
saathis	id (PK), name (Admin only), alias, max_capacity, current_load	Peer supporter roster. Real names restricted to X-Admin-Key.
saathi_chats	id (PK), session_id, saathi_id, role, status, consented_history_transfer	Dual-Saathi connections (PRIMARY / SECONDARY). Independent threads.
saathi_messages	id (PK), saathi_chat_id, sender, text, timestamp	Peer supporter direct messaging log.
appointments	id (PK), counsellor_id, student_name, student_email, slot	Isolated PII. No foreign keys or joins to sessions or chats.

5. Frontend State Management Snippet

```
// App.jsx - Session & Dual Saathi Management
const getSessionId = () => {
  let id = localStorage.getItem("sahara_session_id");
  if (!id) {
    id = "sess-" + crypto.randomUUID();
    localStorage.setItem("sahara_session_id", id);
  }
  return id;
};

// Match Saathis
const matchSaathis = async () => {
  const res = await fetch("http://127.0.0.1:8000/api/saathi/match", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ session_id: getSessionId() })
  });
  const data = await res.json();
  // Returns data.primary and data.secondary
  return data;
};
```